

Desarrollo de componentes.

Caso práctico

En Datalab ya llevan más de tres meses funcionando con su nuevo ERP. El proceso de implantación ha sido largo y complicado, pero han ganado en funcionalidad y productividad. Con el nuevo sistema es posible trabajar mucho más rápido y mejor.

Datalab ha contratado un servicio de mantenimiento con BK Programación. Este servicio incluye la realización de unos módulos nuevos que necesita la empresa. Los responsables de Datalab se han puesto en contacto con Ada para interesarse sobre cómo van los avances de esos módulos a medida. Ada les ha dicho que, una vez terminado el proceso de implantación inicial, ya pueden empezar a desarrollar los módulos a medida.

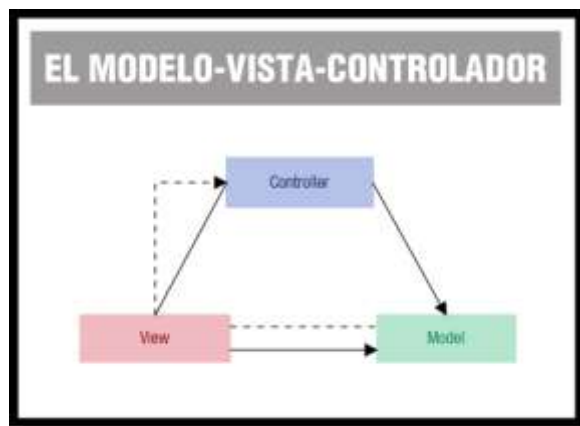
En BK Programación van a desarrollar los nuevos módulos bajo el paradigma de Modelo-Vista-Controlador. Este modelo lo han podido ver frecuentemente en aplicaciones web donde:

- ✓ La vista es la página web y el código que proporciona de datos dinámicos a la página.
- ✓ El modelo es el Sistema de Gestión de Base de Datos.
- ✓ El controlador es el responsable de recibir las peticiones del usuario a través de la vista, consultar datos del modelo, realizar los cálculos necesarios y solicitar nuevas vistas.

1.- Técnicas y estándares. Modelo-Vista-Controlador.

¿Utilizas habitualmente hojas de cálculo? En ese caso, sabrás que, al introducir datos en las celdas, los mismos datos podemos verlos de varias formas. Es posible ver los datos en la hoja de cálculo donde los hemos introducido, o mediante un gráfico que puede ser de barras, circular, etc.

Esos programas implementan el patrón Modelo-Vista-Controlador (MVC), el modelo lo constituyen los datos que hemos introducido en las celdas. Las vistas se encargan de mostrar los datos de la forma que se seleccione.



El MVC divide una aplicación en tres componentes:

- ✓ Los datos de la aplicación (**modelo**).
- ✓ La interfaz del usuario (**vista**).
- ✓ El **controlador**, el cual define la forma en que la interfaz reacciona a la entrada del usuario.

Con esto se consigue separar los datos (modelo) de la interfaz de usuario (vista), de manera que los cambios en la interfaz no afectan a los datos y viceversa, es decir, que los datos pueden ser cambiados sin afectar a la interfaz de usuario.

En Odoo, el MVC se implementa de la siguiente forma:

- ✓ El **modelo** son las tablas de la base de datos.
- ✓ La **vista** son los archivos XML que definen la interfaz de usuario del módulo.
- ✓ El **controlador** son los objetos creados en Python.

Por ejemplo, en el diagrama anterior, las flechas continuas significan que el controlador tiene un acceso completo a la vista y al modelo, y las flechas discontinuas significan un acceso limitado. Las razones de este diseño son las siguientes:

- ✓ **Acceso de modelo a vista:** el modelo envía una notificación a la vista cuando sus datos han sido modificados, con el fin de que la vista pueda actualizar su contenido. El modelo no necesita conocer el funcionamiento interno de la vista para realizar esta operación. Sin embargo, la vista necesita acceder a las partes internas del controlador.
- ✓ **Acceso de vista a controlador:** la razón de que la vista tenga limitado el acceso al controlador es porque las dependencias que la vista tiene sobre el controlador deben ser mínimas ya que el controlador puede ser sustituido en cualquier momento.

1.1.- Especificaciones técnicas para el desarrollo de componentes.

¿Recuerdas en la primera unidad cuando hablábamos de la arquitectura de los sistemas de planificación empresarial? Decíamos que la mayoría están basados en una arquitectura cliente-servidor, este es el caso de Odoo.

Odoo utiliza los protocolos XML-RPC o Net-RPC para comunicación entre cliente y servidor. Básicamente lo que hacen es permitir al cliente hacer llamadas a procedimientos remotos, o sea, ejecutar código de otra máquina. En el caso de XML-RPC, la función llamada, sus argumentos y el resultado se envían por HTTP y son codificadas usando XML. Net-RPC está disponible más recientemente, está basado en funciones en Python y es más rápido.

Odoo funciona sobre un marco de trabajo o framework llamado OpenObject. Este framework permite el desarrollo rápido de aplicaciones (RAD), siendo sus principales elementos los siguientes:

- ✓ ORM: mapeo de bases de datos relacionales a objetos Python.
- ✓ Arquitectura MVC (Modelo Vista Controlador).
- ✓ Diseñador de informes.
- ✓ Herramientas de Business Intelligence y cubos multidimensionales.
- ✓ Cliente de escritorio y cliente web.

1.2.- Especificaciones funcionales para el desarrollo de componentes.

Como sabemos, Odoo se compone de un núcleo y varios módulos dependiendo de las necesidades, por ejemplo:

- ✓ base: es el módulo básico compuesto de objetos como empresas (res.partner), direcciones de empresas (res.partner.address), usuarios (res.user), monedas (res.currency), etc.
- ✓ account: gestión contable y financiera.
- ✓ product: productos y tarifas.
- ✓ purchase: gestión de compras.
- ✓ sale: gestión de ventas.
- ✓ mrp: fabricación y planificación de recursos.
- ✓ crm: gestión de las relaciones con clientes y proveedores.

Por cada módulo existe una carpeta con el nombre del módulo en el directorio **addons** del servidor. Para crear un módulo tendremos que crear una carpeta dentro de ese directorio, y seguir los siguientes pasos:

1. Crear el archivo de inicio de módulo: `__init__.py`
2. Crear el archivo con la descripción del módulo: `__manifest__.py`.
3. Crear los archivos Python con la definición de objetos: `nombre_modulo.py`, por medio de clases del lenguaje Python definimos el modelo y el controlador.
4. Vista o vistas del objeto `nombre_modulo_nombre_objeto.xml`.

En la carpeta **addons** además de los archivos anteriores, puede haber otras subcarpetas como report, wizard, views, que contienen archivos del tipo de objeto utilizado, en este caso, informes, asistentes y accesos directos, datos de prueba, etc.

El archivo `__manifest__.py` debe contener los siguientes valores dentro de un diccionario Python (estructura de datos cerrada con llaves { }):

- ✓ name: nombre del módulo.
- ✓ version: versión del módulo.
- ✓ description: una descripción del módulo.
- ✓ author: persona o entidad que ha desarrollado el módulo.
- ✓ website: sitio web del módulo.
- ✓ license: tipo de licencia del módulo (por defecto GPL).
- ✓ depends: lista de módulos de los que depende el módulo, el módulo base es el más usado ya que en él se definen algunos datos que son necesarios las vistas, informes, etc.
- ✓ init_xml: lista de los archivos XML que se cargarán con la instalación del módulo.
- ✓ installable: determina si el módulo es instalable o no.

Recurso: **Estructura de un módulo para Odoo**

https://wiki.nuxpy.com/index.php/Estructura_de_un_m%C3%B3dulo_para_Odoo

2.- Técnicas de optimización de consultas y acceso a grandes volúmenes de información.

Caso práctico

María está realizando un mantenimiento de la base de datos. Ha observado que determinadas consultas tienen un tiempo de respuesta algo menos rápido que otras y está analizando el código de las mismas. Se ha dado cuenta que hay consultas que no siguen las reglas de optimización, y probablemente ese sea el motivo de que necesiten más tiempo para extraer los datos de la aplicación.

Un aspecto a tener en cuenta a la hora de hacer modificaciones en la aplicación y crear módulos que utilicen nuevos objetos, es que estos objetos se basan en consultas a la información existente en la base de datos. Si deseamos mejorar los tiempos de respuesta del sistema, deberemos crear objetos que utilicen consultas lo más optimizadas posibles. Podemos decir que la optimización es el proceso de modificar un sistema para mejorar su eficiencia o el uso de los recursos disponibles.

Cuando manejamos grandes cantidades de datos, el resultado de una consulta puede tomar un tiempo considerable, obteniendo no siempre una respuesta óptima. Dentro de las técnicas de optimización de consultas podemos encontrar las siguientes:

- ✓ **Diseño de tablas.** A la hora de crear nuevas tablas, asegurarnos de que no hay duplicidad de datos y que se aprovecha al máximo el almacenamiento en las tablas.
- ✓ **Campos.** Es recomendable ajustar al máximo el espacio en los campos para no desperdiciar espacio.
- ✓ **Índices.** Permiten búsquedas a una velocidad notablemente superior, pero no debemos estar incitados a indexar todos los campos de una tabla, ya que los índices ocupan más espacio y se tarda más al actualizar los datos. Dos de las razones principales para utilizar un índice son:
 - Es un campo utilizado como criterio de búsquedas.
 - Es una clave ajena en otra tabla.
- ✓ **Optimizar sentencias SQL.** Existen una serie de reglas para utilizar el lenguaje de consulta y modificación de datos que hay que contemplar. Estas reglas se refieren tanto a la utilización de las sentencias de selección, como a las que realizan alguna inserción o modificación en la base de datos.
- ✓ **Optimizar la base de datos.** También podemos conectarnos en modo comando a la base de datos y utilizar sentencias para optimizar los datos contenidos en la base de datos.

2.1.- Sistemas batch inputs. Generación de programas de extracción y procesamiento de datos.

El término batch-input procede de la utilización en sistemas SAP de un método utilizado para transferir grandes cantidades de datos a un sistema ERP. Existen dos formas de hacer un batch-input:

- ✓ **Método clásico.** Método asíncrono, es decir, que se procesan los datos pero se actualizan más tarde. Tiene la característica de que genera un archivo de mensajes o log para tratarse errores a posteriori.
- ✓ **Método "call transaction".** Método on-line usado para dar de alta rápidamente pocos registros. Es un método síncrono, no genera log es mucho más rápido, pero poco útil para gran cantidad de datos.

Un proceso batch-input se compone de dos fases:

- ✓ **Fase de generación.** Es la fase en que se genera el archivo batch-input con los datos a introducir o modificar.
- ✓ **Fase de procesamiento.** El archivo batch-input se ejecuta, haciéndose efectivas las modificaciones en la base de datos.

3.- Lenguaje proporcionado por los sistemas ERP-CRM.

Caso práctico

María le ha pasado a Juan las especificaciones de los nuevos módulos a realizar a Datalab. Juan le está explicando que el lenguaje de programación a utilizar es Python.

—Es un lenguaje que se suele utilizar para aprender a programar, por su sencillez —comenta Juan—. Sus instrucciones son muy parecidas al lenguaje natural.

—Sí —responde María—, eso había observado.

—Si te parece yo me encargo de programar los módulos, lo que sería hacer el modelo y el controlador.

—De acuerdo -asiente María-, así puedo centrarme en los archivos XML para hacer la vista de los módulos. Hay que tener en cuenta que hay que hacer dos tipos de vista para cada módulo: una de tipo formulario y otra de tipo árbol, para facilitar las búsquedas.

—Muy bien -responde Juan—, pues ¡manos a la obra!

La mayoría de aplicaciones de planificación empresarial están construidas con lenguajes de programación modernos y, en muchos casos, orientados a objetos. Existen aplicaciones ERP que proporcionan lenguajes propietarios, que no son lenguajes estándar y requieren que el programador se forme en ese lenguaje sólo y exclusivamente para manejar el ERP.

El lenguaje de programación utilizado en Odoo es un lenguaje relativamente reciente, el lenguaje Python.

Este lenguaje fue creado por Guido van Rossum a principios de los años 90 en el Centro para las Matemáticas y la Informática (CWI, Centrum Wiskunde & Informatica), en los Países Bajos. Guido sigue siendo el autor principal de Python.

En 2001, se creó la Python Software Foundation (PSF), una organización sin fines de lucro creada específicamente para poseer la propiedad intelectual sobre la licencia y promover el uso del lenguaje. Como miembros patrocinadores de esta organización están Google, Canonical, Sun Microsystems y Microsoft.

La licencia de Python es de código abierto, compatible con GPL. Es un lenguaje muy sencillo de utilizar con una sintaxis muy cercana al lenguaje natural, por lo que se considera uno de los mejores lenguajes para empezar a programar.

Python es utilizado por Google, Yahoo, la NASA o por ejemplo en todas las distribuciones Linux, donde los programas escritos en este lenguaje representan un porcentaje cada vez mayor. Aunque ver en profundidad este lenguaje nos llevaría más tiempo del esperado, vamos a ver algunas de sus características más importantes para poder crear nuestros propios módulos en Odoo.

Para saber más

Si deseas conocer más sobre el lenguaje Python puedes acceder a su web oficial:

[Web oficial del lenguaje Python](https://www.python.org/)

[Como instalar Python](#)

3.1.- Características y sintaxis del lenguaje.

Las principales características de Python son:

- ✓ **Sintaxis muy sencilla**, lo cual facilita un rápido aprendizaje del lenguaje.
- ✓ **Lenguaje interpretado**. Los programas en Python se ejecutan utilizando un programa intermedio llamado intérprete. El código fuente se traduce a un archivo llamado bytecode con extensión .pyc o .pyo, que es el que se ejecutará en sucesivas ocasiones.
- ✓ **Tipado dinámico**. Esta característica quiere decir que los datos no se declaran antes de utilizarlos, sino que el tipo de una variable se determina en tiempo de ejecución, según el dato que se le asigne.
- ✓ **Fuertemente tipado**. Cuando una variable es de un tipo concreto, no se puede usar como si fuera de otro tipo, a no ser que se haga una conversión de tipo. Si la variable es numérica entera, por ejemplo, no podré asignarle un valor real. En otros lenguajes, el tipo de la variable cambiaría para adaptarse al comportamiento esperado, aunque ello puede dar más lugar a errores.
- ✓ **Multiplataforma**. El intérprete de Python está disponible para una gran variedad de plataformas: Linux, Windows, Macintosh, etc.

- ✓ **Orientado a objetos.** Los programas están formados por clases y objetos, que son representaciones del problema en el mundo real. Los datos, junto con las funciones que los manipulan, son parte interna de los objetos y no están accesibles al resto de los objetos. Por tanto, los cambios en los datos de un objeto sólo afectan a las funciones definidas para ese objeto, pero no al resto de la aplicación.

Para hacer el primer programa en Python no necesitamos conocer mucho sobre el lenguaje, por ejemplo, podemos hacer el famoso **"Hola mundo"** con el que se suele comenzar a estudiar casi todos los lenguajes, simplemente con la siguiente línea:
print ('Hola mundo')

Si escribimos esto en el intérprete de Python se ejecutará y se imprimirá por pantalla el mensaje **"Hola mundo"**. ¡Ya tenemos nuestro primer programa! Probémoslo. El intérprete de Python está instalado por defecto en la mayoría de las distribuciones de Linux, para saber si lo tenemos instalado simplemente tenemos que introducir python en un terminal, y nos aparecerá un mensaje con la versión que tenemos instalada y el prompt >>>, que indica que está esperando código del usuario. Podemos salir escribiendo **exit ()**, o con la combinación de teclas **Control + D**.

También podemos guardar el código en un archivo llamado **programa.py**, y ejecutarlo con la orden python **programa.py**, el resultado será que se escribirá la cadena de texto **'Hola mundo'**.

Los comentarios en el lenguaje se escriben con el carácter #, las asignaciones de valores a las variables con el carácter igual (=). Para crear una función utilizamos el siguiente código, hay que tener en cuenta que la sangría del texto sirve para delimitar qué instrucciones van dentro de la función:

```
def nombrefunción(arg1,arg2..):  
    instrucción1  
    instrucción2  
    ...  
    instrucciónN
```

Es decir, en este lenguaje no hay delimitador de sentencia como el punto y como de Java o C, ni delimitador de bloques, como las llaves de estos mismos lenguajes.

3.2.- Declaración de datos. Tipos básicos.

Como ya hemos comentado, en Python no se declaran las variables como tal, sino que se utilizan directamente. Podemos declarar variables de los siguientes tipos básicos:

- ✓ **Números enteros.** Los números enteros son el cero y aquellos números positivos o negativos que no tienen decimales. En Python se pueden representar mediante el tipo int que utiliza 32 o 64 bits, dependiendo del tipo de procesador, o el tipo long, que permite almacenar números de cualquier precisión, estando limitados solo por la memoria disponible en la máquina.

```
# declaración de un número entero
a = 2
# declaración de un número entero largo
a = 2L
```

- ✓ **Números reales.** Son los que tienen la coma decimal. Se expresan mediante el tipo *float*.

```
b = 2.0
```

- ✓ **Números complejos.** Son un tipo de dato especial compuesto por una parte real y una parte imaginaria.

```
complejo = 2.0 + 7.5 j
```

- ✓ **Cadenas.** Es texto encerrado entre comillas simples o dobles.

```
mensaje = "Bienvenido a Python"
```

- ✓ **Booleanos.** Sólo pueden tener dos valores True o False.

De entre los operadores más importantes para trabajar con estos datos podemos destacar los siguientes:

- ✓ **Operadores aritméticos:** suma (+), resta (-), multiplicación (*), división (/), división entera (//), exponente (**), módulo (%).
- ✓ **Operadores booleanos:** and, or, not.
- ✓ **Operadores relacionales:** igual (==), distinto (!=), menor (<), mayor (>), menor igual(<=), mayor o igual (>=).

3.3.- Estructuras de programación. Colecciones.

Python tiene una serie de estructuras de datos que se utilizan ampliamente. Son tipos avanzados de datos, que reciben el nombre de Colecciones, a saber:

- ✓ **Listas.** Son colecciones ordenadas de datos, en otros lenguajes se conocen como arrays o vectores. Pueden tener cualquier tipo de dato.

```
l = [3, True, "mi lista", [ 1 , 2 ] ]
```

Se accede a un elemento concreto de la lista usando el índice de su posición (empezando por cero) entre corchetes.

```
a = l [0]          # a vale 3
b = l [3][1]       # b vale 2
```

- ✓ **Tuplas.** Son parecidas a las listas, con la diferencia que son más ligeras por lo que si el uso es básico se utilizan mejores tuplas. Los elementos van entre paréntesis en lugar de entre corchetes.

```
t = ( "hola", 2, False)
b = t[1]          # b vale 2
```

- ✓ **Diccionarios.** Son colecciones que relacionan una clave y un valor. Para acceder a un valor se utiliza el operador [], igual que para las filas y tuplas, y dentro de él la clave a la que queremos acceder.

```
d = { "Nombre": "Alberto",
      "Apellido" : "Contador",
      "Victorias" : [ 2007 , 2008 , 2009 ] }
x = d ["Nombre"]          # x vale "Alberto"
x = d ["Apellido"]        # x vale "Contador"
x = d ["Victorias"][1]    # x vale 2008
```

Las listas son **objetos mutables**, es decir, podemos modificar cada uno de sus componentes. Las tuplas sin embargo son **no mutables**, es decir, que no podemos modificarlas una vez que se han creado, por ejemplo, si intentamos hacer esto:

```
tupla = ( 1 , 'hola', 3.0)
tupla [1] = 'adios'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> 008
```

El intérprete de Python nos da un error porque las tuplas no se pueden modificar.

3.4.- Sentencias del lenguaje.

La sintaxis de las estructuras de programación en Python es la siguiente:

- ✓ **Estructura condicional.** La sintaxis más común de la estructura condicional es un `if` seguido de la condición a evaluar, seguida de dos puntos, y la siguiente línea sangrada, para indicar el código que queremos que se ejecute si se cumple la condición. En caso de que no se cumpla la condición, ejecutamos la sentencia detrás del `else`:

```
if (numero / 2 == 0):  
    print ("El numero es par")  
else:  
    print ("El numero es impar")
```

- ✓ **Bucles.** Nos permiten ejecutar un fragmento de código un número de veces, o mientras se cumpla una determinada condición.

```
while)
```

El bucle *while* (mientras) repite todo su bloque de código mientras la expresión evaluada sea cierta. Detrás de la condición hay que poner dos puntos, y debajo las sentencias que van dentro del `while`.

```
num = 1  
while num <= 10:  
    print (num)  
    num += 1
```

El funcionamiento es sencillo, el programa evalúa la condición, y si es cierta, ejecuta el código que hay dentro del *while*, cuando acaba vuelve a evaluar la condición y si es cierta ejecuta nuevamente el código, y así hasta que la condición devuelva *false*, en cuyo caso, el programa continuaría ejecutando las instrucciones siguientes al bucle. `for ... in`

La sentencia `for` se utiliza para recorrer secuencias de elementos. Si utilizamos un tipo secuencia, como es la lista en el siguiente ejemplo:

```
milista = ["Maria", "Pepe", "Juan"]  
for elemento in milista:  
    print (elemento )
```

Vemos que el resultado es:

```
Maria
Pepe
Juan
```

Lo que hace el bucle es que para cada elemento que haya en la secuencia, ejecuta las líneas de código que tenga dentro, en este caso la sentencia **print**. Lo que hace la cabecera del bucle es obtener el siguiente elemento de la secuencia **lista** y almacenarlo en una variable de nombre **elemento**. Por esta razón en la primera iteración del bucle **elemento** valdrá "Maria", en la segunda "Pepe", y en la tercera "Juan".

3.5.- Llamadas a funciones.

Una función es un conjunto de instrucciones o programa que realiza una tarea específica y que devuelve un valor. Las funciones nos ayudan a reutilizar código, y a dividir el programa en partes más pequeñas con objeto de hacerlo más organizado, legible y más fácil de depurar.

```
def cuadrado ( numero ) :
    print ( numero ** 2 )
```

Con el código anterior estamos declarando la función **cuadrado** que, como su nombre indica, calcula el cuadrado de un número. El número que aparece entre paréntesis es el valor que le pasamos a la función para que pueda calcular su cuadrado, y recibe el nombre de parámetro. Las funciones pueden tener uno, varios o ningún parámetro.

Cuando ejecutamos la función se dice que estamos haciendo una llamada a la función, y lo hacemos utilizando su nombre y los parámetros entre paréntesis:

```
cuadrado (4)
```

El resultado de llamar a la función será que se imprimirá por pantalla el valor **16**.

Las funciones pueden o no devolver un valor como resultado de su ejecución. Podemos transformar el ejemplo anterior para que devuelva el resultado en lugar de mostrarlo por pantalla. El código sería el siguiente:

```
def cuadrado ( numero ) :
    return numero ** 2
```

Y llamaríamos a la función con la siguiente instrucción:

```
print (cuadrado(4))
```

El resultado sería el mismo, sólo que ahora la función devuelve el valor, y ese valor es el que muestra por pantalla la instrucción **print**.

3.6.- Clases y objetos.

Hasta ahora hemos visto la sintaxis básica de Python. Pero para entender bien cómo está formado un módulo en Odoo, necesitamos saber también qué son las clases y los objetos. Si has programado antes con un lenguaje orientado a objetos, no tendrás ningún problema en entender la definición de ambos conceptos.

Un programa orientado a objetos está formado por clases y objetos. Cuando tenemos un problema que resolver, lo dividimos en una serie de objetos y son esos objetos los que describimos en nuestro programa. El programa consiste en una serie de interacciones entre esos objetos. Las clases son plantillas a partir de las cuales se crean los objetos, cuando creamos un objeto a partir de una clase se dice que ese objeto es una instancia de la clase.

Las clases tienen una serie de atributos o campos que las definen, así como una serie de métodos para hacer operaciones sobre esos campos. Cuando creamos un objeto le damos valor a esos atributos. En Python las clases se definen mediante la palabra clave `class` seguida del nombre de la clase, dos puntos (`:`) y a continuación, sangrado, el cuerpo de la clase.

Por ejemplo, vamos a crear una clase `Persona` que tiene como atributos el nombre y la edad:

```
class persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
    def mayordeedad(self):
        if self.edad > 18:
            print ("Es mayor de edad" )
        else:
            print ("No es mayor de edad" )
```

En la definición de clase anterior vemos que estamos definiendo un método `__init__`. El nombre de este método no es casual, sino que se utiliza para inicializar el objeto. Cuando instanciamos un objeto a partir de la clase, será el primer método que se ejecutará. También vemos que los dos métodos de la clase tienen como primer argumento `self`. Este argumento sirve para referirnos al objeto actual, y se utiliza para poder acceder a los atributos y métodos del objeto.

Para crear un objeto de la clase utilizamos la siguiente instrucción:

```
empleado = persona("Javier",32)
```

Ahora que ya hemos creado nuestro objeto, podemos acceder a sus atributos y métodos mediante la sintaxis objeto.atributo y objeto.método:

```
print (empleado.nombre, "tiene", empleado.edad)
```

3.7.- Módulos y paquetes.

Cuando salimos del intérprete y volvemos a entrar empezamos desde el principio, si habíamos creado alguna variable o programa éste desaparece y tenemos que volver a escribir el código. Para conservar los programas que hacemos, podemos crear archivos con extensión .py, de manera que siempre tengamos el código disponible para ejecutarlo con el intérprete y que no sea necesario volver a introducirlo.

Por otra parte, los programas a veces son muy grandes y al dividirlos en partes se hacen más fáciles de mantener. Se agrupa el código relacionado y se guarda en archivos diferentes. Cada archivo es un módulo en Python. En otras palabras, un módulo es un archivo que contiene definiciones y declaraciones de Python.

Las razones para utilizar módulos son principalmente las siguientes:

- ✓ Hacer el código más fácil de mantener.
- ✓ Reutilización de código, por ejemplo, una función que queramos utilizar en varios programas.

Si queremos utilizar un módulo en otro archivo, tenemos que importarlo con la siguiente instrucción:

```
import nombre_modulo
```

Un paquete es una colección de módulos relacionados. Mientras los módulos ayudan a organizar el código, los paquetes ayudan a organizar los módulos. A nivel físico, los módulos son los archivos, y los paquetes son los directorios.

Para hacer que Python trate a un directorio como un paquete es necesario crear un archivo **__init__.py** en dicha carpeta. Para hacer que un módulo pertenezca a un paquete hay que copiar el archivo **nombre_modulo.py** que define el módulo en el directorio del paquete. Para importar paquetes se utiliza la sentencia **import** igual que con los módulos.

3.8.- Librerías de funciones (APIs).

Una **API**, o **Biblioteca de Clases**, es un conjunto de clases útiles que tienen a disposición los programadores para utilizar en sus programas. Python proporciona una API estándar, cuyo código fuente está a libre disposición para las principales plataformas desde el sitio web de Python. También es posible acceder a muchos módulos libres de terceros en la misma página web, así como a programas, herramientas y documentación adicional.

En el enlace expresado más abajo encontrarás una referencia a toda la Biblioteca estándar de módulos de Python, entre los que destacan:

- ✓ **os**: provee de diferentes funciones del sistema operativo, como creación de archivos y directorios, leer o escribir de un archivo, manipular rutas, etc.
- ✓ **sys**: permite acceder a información sobre el intérprete de Python, como por ejemplo el prompt del sistema, datos de licencia y en general cualquier parámetro o variable que tenga que ver con el intérprete.
- ✓ **datetime**: módulos para la manipulación de fechas y horas.
- ✓ **math**: funciones matemáticas.

3.9.- Inserción, modificación y eliminación de datos en objetos.

En este apartado vamos a ver cómo están estructurados los módulos y cómo crear un primer módulo sencillo que cree un objeto en la aplicación para insertar, modificar o eliminar datos en la base de datos.

Los módulos de Odoo se guardan dentro de la carpeta addons.

Lo que tenemos que hacer es crear una carpeta para nuestro módulo dentro del directorio addons. Los archivos que tiene que contener la carpeta son:

- ✓ **__init__.py**. Necesario para que la carpeta se trate como un paquete en Python, contiene los import de cada archivo del módulo que contenga código Python, en este caso, el archivo **nombre_modulo.py**.
- ✓ **__manifest__.py**. Contiene un diccionario Python con la descripción del módulo, como quién es el autor, la versión del módulo o cuáles son los otros módulos de los que depende.
- ✓ **nombre_modulo.py**. En este archivo creamos la clase definiendo los campos que va a tener. Al crear la clase estamos creando el modelo (una tabla en la base de datos) y también estamos creando el controlador, porque cuando creamos una clase estamos definiendo el comportamiento que tiene.
- ✓ **nombre_modulo_view.xml**. En este archivo definimos la vista de nuestro módulo, o del objeto que va a crear nuestro módulo. Para crear este archivo necesitamos tener ciertos conocimientos de XML, o bien coger una vista de otro objeto y a partir de ella crear la del nuevo objeto.

También existe otra forma de crear módulos sin necesidad de ningún desarrollo, que es utilizando el módulo **base_module_record**. Si instalamos este módulo, tendremos la opción de grabar todas las acciones que realicemos en la aplicación. Si has utilizado alguna vez las macros en un procesador de textos o en una hoja de cálculo sabrás de lo que estamos hablando. El módulo **base_module_record** funciona de forma parecida. Por ejemplo, podemos crear un nuevo objeto y asignarle un menú, todo ello se grabaría en un archivo comprimido. Realmente

ese archivo comprimido lo que va a contener es un módulo, con todos los archivos indicados anteriormente. Si con posterioridad cogemos esos archivos y los copiamos a la carpeta **addons**, podremos instalarlos en la aplicación como cualquier otro módulo. Al instalarlo, el resultado sería que aparecería ese nuevo objeto y menú en nuestra aplicación.

Debes conocer

En el siguiente enlace tienes un ejemplo de creación de un módulo:

[Crear un módulo en Odoo](#)

4.- Entornos de desarrollo y herramientas de desarrollo en sistemas ERP-CRM.

Caso práctico

Juan está buscando un entorno de desarrollo para la creación de los módulos. Ya ha empezado a programar, el código de los módulos es bastante extenso y le resulta tedioso tener que trabajar con el intérprete. Necesita un entorno de desarrollo que le ayude con la sintaxis del lenguaje, y a depurar los programas.

La herramienta más importante cuando estamos programando siempre es un buen entorno de desarrollo. Si queremos una respuesta rápida y el programa es pequeño podemos utilizar el intérprete, como hemos venido haciendo hasta ahora. Sin embargo, cuando el programa va siendo más grande, es mucho más cómodo utilizar un entorno de desarrollo desde el cual podemos hacer todas las operaciones de manera gráfica.



Un entorno de desarrollo es un programa que incluye todo lo necesario para programar en uno o varios lenguajes. Además de un editor de textos especializado como herramienta central, incluyen navegadores de archivos, asistentes de compilación, depuradores, etc.

Algunos editores:

[Entorno de desarrollo para Odoo.](#)

4.1.- Depuración de un programa.

Las herramientas de depuración cuando estamos desarrollando un programa ayudan a detectar errores en el código, mostrando información de qué tipo de error es o porqué se produce. Las herramientas de depuración suelen formar parte de los entornos de desarrollo.

El lenguaje Python incorpora un depurador dentro de su Biblioteca de módulos estándar llamado **pdb**. Soporta puntos de ruptura y ejecución paso a paso del código, inspeccionar valores de variables y otras opciones de depuración. Sin embargo, como en todos los lenguajes, si utilizamos una herramienta gráfica para el proceso de depuración, seguramente ahorraremos mucho tiempo.

Dentro del entorno de desarrollo IDLE podemos depurar nuestros programas. Este entorno está disponible para cualquier distribución y sistema operativo. Para activar el módulo de depuración hacemos clic en el menú Debug/Debugger. Una vez que esté el modo de depuración activado el funcionamiento es como el de cualquier depurador. Con el botón derecho establecemos una parada en la sentencia del código que queremos evaluar, de manera que al ejecutar el programa el depurador se parará en esa instrucción, y podremos analizar el valor de las variables en ese punto.

Las acciones que podemos realizar con el depurador son las siguientes:

- ✓ Go: hace que la ejecución del programa continúe hasta que encuentre un punto de ruptura.
- ✓ Step: va ejecutando el programa línea a línea.
- ✓ Over: hace que la sentencia actual sea completamente ejecutada sin parar en ninguna función anidada.
- ✓ Out: hace que se compute desde el punto actual hasta el fin de la función actual y que esta se termine.
- ✓ Quit: finaliza la ejecución del programa.

4.2.- Manejo de errores.

Cuando nuestro programa falla, se genera un error en forma de excepción. Las excepciones son errores detectados por el intérprete durante la ejecución del programa. Por ejemplo, podemos intentar dividir por cero o acceder a un archivo que no existe, entonces el programa genera una excepción avisando al usuario de que se ha producido un error. Nuestra labor como programadores es escribir el código de manera que se contemplen esas posibles excepciones, y se actúe en consecuencia. Si nuestro programa no contempla el manejo de excepciones, cuando ocurra alguna lo que pasará es que se generará el error y el programa dejará de ejecutarse o no tendrá el funcionamiento esperado.



En Python se utiliza una construcción **try-except** para capturar y tratar las excepciones. Dentro del bloque try se sitúa el código que creemos que podría producir una excepción, y dentro del bloque **except**, escribimos el código que se ejecutará si se produce dicha excepción.

Veamos un pequeño programa que lanzaría una excepción al intentar hacer una división entre 0:

```
def dividir(a, b):  
    return a / b  
dividir(3,0)
```

Si lo ejecutamos en el intérprete obtenemos la siguiente salida de error:

```
ZeroDivisionError: integer division or modulo by zero
```

Lo que nos dice el mensaje es que es una excepción, `ZeroDivisionError`, y la descripción del error: "integer division or modulo by zero" (módulo o división entera entre cero).

Si escribimos el mismo programa, pero teniendo en cuenta el manejo de excepciones, el código quedaría como sigue:

```
try:  
    def dividir(a, b):  
        return a / b  
    dividir(1,0)  
except:  
    print ("Ha ocurrido un error")
```

Para saber más

Si quieres aprender más sobre excepciones puedes consultar el siguiente tutorial de Python:

[Información sobre excepciones en Python](#)

5.- Formularios e informes en sistemas ERP-CRM.

Caso práctico

María está repasando conceptos de XML para generar las vistas del modelo. Concretamente está analizando cómo están formadas las vistas de los módulos que vienen instalados con la aplicación. Lo primero que ha hecho es hacerse una plantilla XML, donde ha incluido todo el código necesario para crear un menú, una acción y dos vistas formulario y árbol para el objeto que incluye el módulo. A partir de ahora utilizará esa plantilla, y sólo tendrá que cambiar los nombres de los elementos para generar vistas nuevas de un objeto.

Los **formularios** constituyen la interfaz de un módulo. Una vez creado el objeto del módulo, o sea, el archivo **nombre_modulo.py**, lo que debemos hacer es crear los menús, acciones, vistas (formulario u otro tipo) para poder interactuar con el objeto. Estos elementos se describen en el archivo **nombre_modulo_view.xml**.

Decir que es importante recordar que el nombre del fichero XML se encuentra listado en el atributo **update_xml** del **fichero__manifest__.py**, **__openerp__.py** a partir de la versión 5.0, del módulo.

Los **informes** pueden ser estadísticos, para los que tenemos el módulo **base_report_creator** que permite crear listados, informes y gráficos por pantalla, y pueden ser informes impresos, en cuyo caso existen diversas herramientas para generarlos.

5.1.- Arquitectura de formularios e informes. Elementos.

Los formularios y demás vistas de la aplicación son contruidos de forma dinámica por la descripción XML de la pantalla del cliente. Podemos decir que el lenguaje XML es un metalenguaje que utiliza etiquetas del tipo **etiqueta_contenido_fin** etiqueta para expresar información estructurada.

Una etiqueta consiste en una marca hecha en el documento, que hace referencia a un elemento o pedazo de información. Las etiquetas tienen la forma, donde nombre es el nombre del elemento que se está señalando.

Cada tipo de registro, hace referencia a un objeto diferente. Los registros se definen con la etiqueta **<record> </record>**, que indican el inicio y fin de la descripción del registro. Dentro van los campos **field** que se utilizan para definir las características del registro. Siguiendo la estructura anterior, un ejemplo de creación de **vista de tipo formulario** sería el siguiente:

```

<!-- Formulario -->
<record model="ir.ui.view" id="citasjfsp.list_form">
  <field name="name">Detalle de Cita</field>
  <field name="model">citasjfsp.citasjfsp</field>
  <field name="arch" type="xml">
<form string="Detalle de Cita">
  <sheet>
    <div class="oe_title">
      <label for="autor" class="oe_edit_only" string="Escribe tu cita:">Cita</label>
      <h1><field name="cita"/></h1>
    </div>
    <separator string="Datos de la cita:" colspan="2" />
    <group colspan="2" col="2">
      <field name="autor"/>
      <field name="fecha_visualizacion"/>
      <field name="orden"/>
    </group>
  </sheet>
</form>
  </field>
</record>

```